

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования

**«Пермский национальный исследовательский политехнический
университет»**

Электротехнический факультет

Выпускающая кафедра: Информационные технологии и
автоматизированные системы (ИТАС)

Направление подготовки: 09.03.04 Программная инженерия

ОТЧЕТ

Лабораторная работа №4

«Простое наследование. Принцип подстановки.»

По дисциплине «Основы алгоритмизации и программирования»

Вариант 15

Выполнил: студент группы РИС-25-2Б

Ильинов Фёдор Михайлович

Принял: Доц. Полякова О. А.

Пермь 2026

ПОСТАНОВКА ЗАДАЧИ

1. Определить пользовательский класс.
2. Определить в классе следующие конструкторы: без параметров, с параметрами, копирования.
3. Определить в классе деструктор.
4. Определить в классе компоненты-функции для просмотра и установки полей данных (селекторы и модификаторы).
5. Перегрузить операцию присваивания.
6. Перегрузить операции ввода и вывода объектов с помощью потоков.
7. Определить производный класс.
8. Написать программу, в которой продемонстрировать создание объектов и работу всех перегруженных операций.
9. Реализовать функции, получающие и возвращающие объект базового класса.

Продемонстрировать принцип подстановки.

Базовый класс:

ЧЕЛОВЕК (PERSON)

Имя (name) – string

Возраст (age) – int

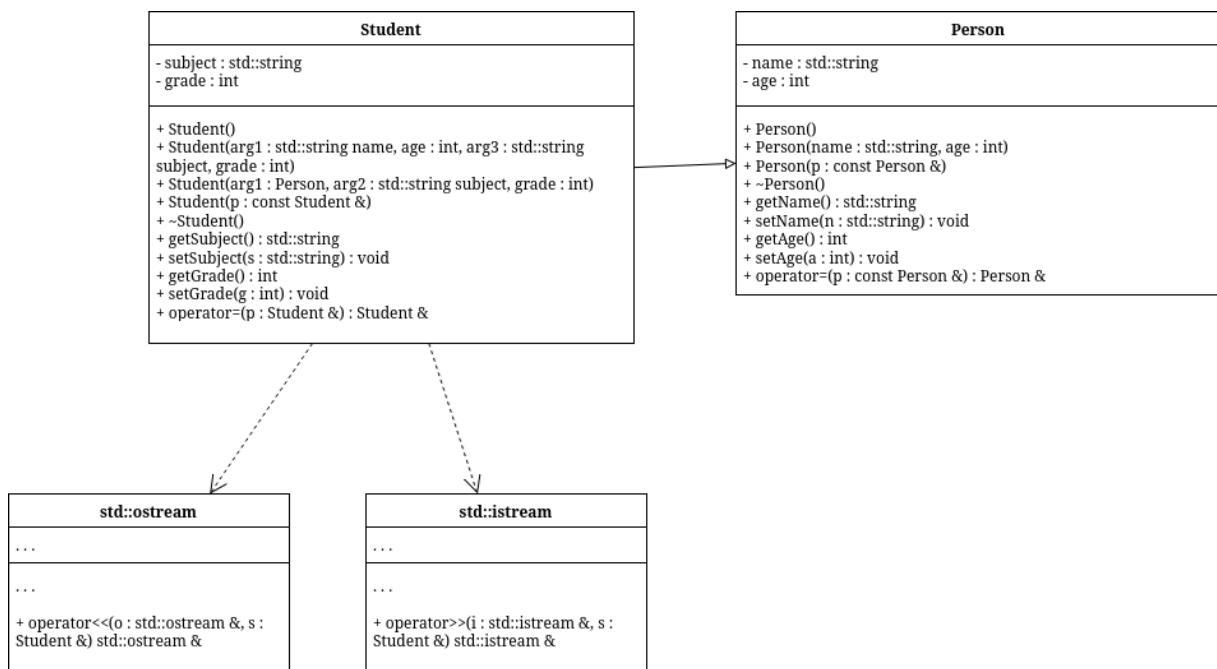
Определить методы изменения полей.

Создать производный класс STUDENT, имеющий поля Предмет – string и Оценка – int. Определить методы изменения полей и метод, выдающий сообщение о неудовлетворительной оценке.

АНАЛИЗ.

1. Согласно принципам инкапсуляции поля класса должны быть приватными, а описанные в постановке методы — публичными.
2. Для работы с приватными полями реализуем гетеры и сетеры.
3. Для объявления переменных заданного класса реализуем конструкторы: обычный (принимает на вход значения полей класса), копирования и «пустой конструктор», выставляющий значения по умолчанию.

UML-ДИАГРАММА



КОД Person.h

```
//  
// Created by localuser on 4/8/26.  
//  
  
#ifndef ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_PERSON_H  
#define ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_PERSON_H  
#include <string>  
  
class Person {  
    std::string name;  
    int age;  
  
public:  
    Person();  
    Person(std::string, int);  
    Person(Person&);  
    ~Person();  
    const std::string getName();  
    void setName(std::string);  
    const int getAge();  
    void setAge(int);  
    Person &operator=(Person const &p);  
    friend std::ostream &operator<<(std::ostream &, const Person &);  
    friend std::istream &operator>>(std::istream &, Person &);  
};  
  
#endif //ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_PERSON_H
```

КОД Person.cpp

```
//  
// Created by localuser on 4/8/26.  
//  
  
#ifndef ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_PERSON_H  
#define ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_PERSON_H  
#include <string>  
  
class Person {  
    std::string name;
```

```

    int age;

public:
    Person();

    Person(std::string, int);

    Person(Person&);

    ~Person();

    const std::string getName();

    void setName(std::string);

    const int getAge();

    void setAge(int);

    Person &operator=(Person const &p);

    friend std::ostream &operator<<(std::ostream &, const Person &);

    friend std::istream &operator>>(std::istream &, Person &);
};

#endif //ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_PERSON_H

```

KOД Student.h

```

#ifndef ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_Student_H
#define ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_Student_H
#include <string>
#include "Person.h"

class Student : public Person {
    std::string subject;
    int grade;

public:
    Student();

```

```
Student(std::string name, int age, std::string subject, int grade);

Student(Person, std::string subject, int grade);

Student(Student&);

~Student();

std::string getSubject();

void setSubject(std::string);

int getGrade();

void setGrade(int);

Student &operator=(Student &p);

friend std::ostream &operator<<(std::ostream &, Student &);

friend std::istream &operator>>(std::istream &, Student &);
};

#endif //ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_Student_H
```

КОД Student.cpp

```
//  
// Created by localuser on 4/8/26.  
//  
  
#include "Student.h"  
  
#include <iostream>  
  
Student::Student() {  
    subject = "Unknown";  
    grade = 5;  
}  
  
Student::Student(std::string name, int age, std::string subject, int grade) : Person(name, age) {  
    this->subject = subject;  
    this->grade = grade;  
}  
  
Student::Student(Person p, std::string subject, int grade) : Person(p){  
    this->subject = subject;  
    this->grade = grade;  
}  
  
Student::Student(Student & p) {  
    subject = p.subject;  
    grade = p.grade;  
}  
  
Student::~Student() {  
    subject = "Unknown";  
    grade = 5;  
}  
  
std::string Student::getSubject() {  
    return subject;  
}  
  
void Student::setSubject(std::string s) {  
    subject = s;  
}  
  
int Student::getGrade() {  
    return grade;  
}
```

```

}

void Student::setGrade(int a) {
    grade = a;
}

Student &Student::operator=(Student &p) {
    this->setName(p.getName());
    this->setAge(p.getAge());
    subject = p.subject;
    grade = p.grade;
    return *this;
}

std::ostream &operator<<(std::ostream &stream, Student &p) {
    std::cout << p.getName() << " : " << p.getAge() << " / " << p.getSubject() << " : " << p.getGrade()
<< '\n';
    return stream;
}

std::istream &operator>>(std::istream &stream, Student &p) {
    std::cout << "name? ";
    std::string s;
    stream >> s;
    p.setName(s);
    std::cout << "age? ";
    int a;
    stream >> a;
    p.setAge(a);
    std::cout << "subject? ";
    stream >> p.subject;
    std::cout << "grade? ";
    stream >> p.grade;
    return stream;
}

```

КОД main.cpp

```

//
// Created by localuser on 4/8/26.
//

```

```

#include "Student.h"

```

```

#include <iostream>

```



```

Student::Student() {
    subject = "Unknown";
    grade = 5;
}

Student::Student(std::string name, int age, std::string subject, int grade) : Person(name, age) {
    this->subject = subject;
    this->grade = grade;
}

Student::Student(Person p, std::string subject, int grade) : Person(p){
    this->subject = subject;
    this->grade = grade;
}

Student::Student(Student & p) {
    subject = p.subject;
    grade = p.grade;
}

Student::~Student() {
    subject = "Unknown";
    grade = 5;
}

std::string Student::getSubject() {
    return subject;
}

void Student::setSubject(std::string s) {
    subject = s;
}

int Student::getGrade() {
    return grade;
}

void Student::setGrade(int a) {
    grade = a;
}

Student &Student::operator=(Student &p) {
    this->setName(p.getName());
    this->setAge(p.getAge());
}

```

```

    subject = p.subject;
    grade = p.grade;
    return *this;
}

std::ostream &operator<<(std::ostream &stream, Student &p) {
    std::cout << p.getName() << " : " << p.getAge() << " / " << p.getSubject() << " : " << p.getGrade()
<< '\n';
    return stream;
}

std::istream &operator>>(std::istream &stream, Student &p) {
    std::cout << "name? ";
    std::string s;
    stream >> s;
    p.setName(s);
    std::cout << "age? ";
    int a;
    stream >> a;
    p.setAge(a);
    std::cout << "subject? ";
    stream >> p.subject;
    std::cout << "grade? ";
    stream >> p.grade;
    return stream;
}

```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Alex : 20

Alex : 20 / Math : 5

John : 20 / Math : 5

Bob : 21

ВОПРОСЫ

1. Для чего используется механизм наследования?

С помощью наследования может быть создана иерархия классов, которые совместно используют код и интерфейсы.

2. Каким образом наследуются компоненты класса, описанные со спецификатором public?

private - не доступен

public - public

protected - protected

3. Каким образом наследуются компоненты класса, описанные со спецификатором private?

private - не доступен

public - private

protected - private

4. Каким образом наследуются компоненты класса, описанные со спецификатором protected?

private - не доступен

public - protected

protected - protected

5. Каким образом описывается производный класс?

class Student : Person

6. Наследуются ли конструкторы?

Нет

7. Наследуются ли деструкторы?

Деструкторы не наследуются как функции, но они автоматически вызываются по цепочке наследования

8. В каком порядке конструируются объекты производных классов?

Сначала базовые, потом производные

9. В каком порядке уничтожаются объекты производных классов?

Сначала производные, потом базовые

10. Что представляют собой виртуальные функции и механизм позднего связывания?

Виртуальная функция позволяет переопределить метод базового класса в производном, чтобы затем, на этапе выполнения

программы вызывался метод не для базового класса (раннее связывание), а

производного, в зависимости от реального

объекта.

11. Могут ли быть виртуальными конструкторы? Деструкторы?

Нет

12. Наследуется ли спецификатор virtual?

По определению да

13. Какое отношение устанавливает между классами открытое наследование?

Открытое (public) наследование устанавливает отношение «является» (is-a) между классами, при котором объект производного класса может использоваться везде, где ожидается объект базового класса.

14. Какое отношение устанавливает между классами закрытое наследование?

Закрытое (private) наследование означает наследование реализации, а не типа

15. В чем заключается принцип подстановки?

Объекты производного класса должны быть взаимозаменяемы с объектами базового класса без нарушения корректности программы.

16. Имеется иерархия классов:

```
class Student
{
    int age;
    public:
    string name;
    ...
};
class Employee : public Student
{
    protected:
    string post;
    ...
};
class Teacher : public Employee
{
    protected: int stage;
    ...
};
Teacher x;
```

Какие компонентные данные будет иметь объект x?

Все, но без доступа к age

17. Для классов Student, Employee и Teacher написать конструкторы без параметров.

18. Для классов Student, Employee и Teacher написать конструкторы с параметрами.

19. Для классов Student, Employee и Teacher написать конструкторы копирования.
20. Для классов Student, Employee и Teacher определить операцию присваивания.

```
#include <string>
```

```
class Student {
```

```
    int age;
```

```
public:
```

```
    std::string name;
```

```
    Student() {
```

```
        age = 0;
```

```
        name = "";
```

```
    }
```

```
    Student(int a, const std::string &n) {
```

```
        age = a;
```

```
        name = n;
```

```
    }
```

```
    Student(const Student &s) {
```

```
        age = s.age;
```

```
        name = s.name;
```

```
    }
```

```
    Student &operator=(const Student &other) {
```

```
        if (this != &other) {
```

```
            age = other.age;
```

```
            name = other.name;
```

```
        }
```

```
        return *this;
```

```
    }
```

```
};
```

```
class Employee : public Student {
```

```
protected:
```

```
    std::string post;
```

```
public:
```

```
    Employee() : Student() {
```

```
        post = "";
```

```
    }
```

```
    Employee(int a, const std::string &n, const std::string &p) : Student(a, n) {
```

```
        post = p;
```

```
    }
```

```

Employee(const Employee &e) : Student(e) {
    post = e.post;
}

Employee &operator=(const Employee &e) {
    if (this != &e) {
        Student::operator=(e);
        post = e.post;
    }
    return *this;
}
};

class Teacher : public Employee {
protected:
    int stage;

public:
    Teacher() : Employee() {
        stage = 0;
    }

    Teacher(int a, const std::string &n, const std::string &p, int s)
        : Employee(a, n, p) {
        stage = s;
    }

    Teacher(const Teacher &t) : Employee(t) {
        stage = t.stage;
    }

    Teacher &operator=(const Teacher &t) {
        if (this != &t) {
            Employee::operator=(t);
            stage = t.stage;
        }
        return *this;
    }
};

```